

# FreshTrack

How it was built — frontend, backend, database, and the custom-trained freshness model

---

## 1. The Big Picture

---

*Think of the whole app like a restaurant. The **frontend** is the dining room — what the customer (user) sees and interacts with. The **backend** is the kitchen — does the real work, hidden from view. The **database** is the pantry/walk-in fridge — where everything is actually stored long-term. The **AI** is a smart assistant chef who can look at a photo and give an opinion.*

The golden rule that ties it together: **the frontend never talks to the database or to AI providers directly.** Every sensitive or heavy operation goes through the backend first. This is the single most important architectural fact because it demonstrates the difference between a toy project and something built with real security thinking

## 2. Frontend — what the user sees and touches

---

### React + TypeScript

**React** is a JavaScript library for building UIs out of small, reusable pieces called **components** — think LEGO blocks. A button, a recipe card, the bottom navigation bar: each is written once as a component and reused everywhere it's needed. When data changes (say, a pantry item's quantity updates), React automatically re-renders only the parts of the screen that actually depend on that data, instead of redrawing the whole page.

**TypeScript** is JavaScript with a type system layered on top — you declare what kind of data everything is (a number, a string, a specific shape of object) and the compiler checks it before the app ever runs. This catches an entire category of bugs (like passing the wrong shape of data into a function) at write-time instead of as a runtime crash a user hits later.

### Routing — TanStack Router (file-based)

Routing decides which screen shows for which URL. This project uses **file-based routing**: the file's name in the `src/routes/` folder directly determines the page and its URL — a file called `_shell.pantry.tsx`

automatically becomes the Pantry page at `/pantry`. There's no manual "if URL equals X, render Y" logic to maintain; the folder structure is the routing table.

## Styling — Tailwind CSS + shadcn/ui

Instead of separate CSS files, **Tailwind CSS** lets you style elements with small utility class names directly in the markup (e.g. a class like `rounded-2xl` or `text-muted-foreground`). **shadcn/ui** provides a set of pre-built, accessible components (dialogs, switches, buttons) — but unlike a typical library, the actual component code is copied into the project rather than hidden in a package, so it can be freely customized.

## State & data fetching — TanStack Query

Whenever the frontend needs data from the backend (the pantry list, today's recipe, notification settings), it uses **TanStack Query**, which handles caching, background refetching, and loading/error states automatically. This is why, for example, saving a new pantry item can instantly show up in the list — the relevant query gets invalidated and refetched without writing manual refresh logic everywhere.

# 3. Backend

---

*The backend is the part of the app that runs on a server, not on the user's phone or browser. Anything involving a secret (an API key), sensitive logic (checking who's allowed to see what), or a call to an external service goes through here. The user's device asks a question; the backend is the only one trusted to actually go get the answer.*

## Server functions — the core backend pattern

This app is built with **TanStack Start**, and its main backend building block is `createServerFn`. This lets you write a function that *looks* like a normal function the frontend can just call directly — but it actually only ever runs on the server. The framework automatically wires up the network request underneath, so there's no manually writing a REST API endpoint and a matching fetch call by hand.

```
export const detectGroceries = createServerFn({ method: "POST" })
  .middleware([requireSupabaseAuth])
  .inputValidator((input) => ImageInput.parse(input))
  .handler(async ({ data, context }) => {
    // this code only ever runs on the server
    const result = await callVision(data.image, systemPrompt, instruction);
    return { items: result };
  });
```

Three things are happening in that pattern, and each one matters:

- **middleware** — runs before the handler. Here it's `requireSupabaseAuth`, which checks the user actually has a valid session before anything else runs, and hands the handler a verified `userId`. No logged-out request ever reaches the real logic.

- **inputValidator** — checks the shape of incoming data (using a library called Zod) before the handler trusts it. If the frontend sent garbage, it's rejected here, not deeper in the code.
- **handler** — the actual logic. This is the only place allowed to hold API keys, talk to the database with elevated trust, or call an external AI provider.

## Real examples from this app

**Photo scanning** (`vision.functions.ts`) — `detectGroceries` takes a base64-encoded photo, sends it to Google's Gemini model through Lovable's AI Gateway with a system prompt instructing it to act as a grocery-identifying vision model, and returns structured JSON: product name, category, estimated shelf life, and a 0–1 freshness score with a short reason. `parseReceipt` works the same way but is prompted specifically to OCR a receipt photo and return a list of purchased line items.

**AI recipes and the assistant** (`ai.functions.ts`) — `suggestRecipes` takes the user's chosen pantry ingredients (or none, for "Surprise Me") and asks the model for recipes that use them. `getDailyRecipe` is scoped to run once per user per day — it checks the database first, and only calls the AI model if nothing's been generated yet for today, then saves the result so every device sees the same recipe. `askAssistant` is a chat function with a system prompt restricting it to pantry/recipe/shopping topics, refusing anything unrelated.

**Permission tiers** (`admin.functions.ts`) — This demonstrates real access-control thinking, rather than simply checking whether a user is logged in:

```
async function assertOwner(context) {
  if (context.userEmail !== OWNER_EMAIL) throw new Error("Forbidden");
}
async function assertAdmin(context) {
  const isOwner = context.userEmail === OWNER_EMAIL;
  const hasAdminRole = await checkUserRole(context.userId, "admin");
  if (!isOwner && !hasAdminRole) throw new Error("Forbidden");
}
```

`assertOwner` only ever passes for one specific, hardcoded account. `assertAdmin` passes for the owner or anyone granted the admin role. Functions that expose other users' emails or account-level data require `assertOwner`; functions for routine moderation (approving a submitted product, for example) only require `assertAdmin`. This means even if an admin account were compromised, it still can't see other users' personal data — the permission boundary is enforced in code, not just assumed.

**Data hooks** (`data.ts`) — most everyday CRUD (create/read/update/delete) for pantry items, shopping list entries, and settings goes through TanStack Query hooks that wrap Supabase client calls, each one scoped to the signed-in user's own rows.

## 4. Database

---

*If the backend is the kitchen, the database is the walk-in fridge and pantry combined — the actual place everything is stored. But this fridge has a rule built into its walls, not just a sign on the door: it will only hand food to the person whose name is on it, no matter who's asking.*

## Supabase = Postgres + Auth + Storage, managed

**Supabase** is a hosted platform built around **PostgreSQL** — a mature, industry-standard relational database that stores data in tables made of rows and columns, similar in concept to a very powerful spreadsheet, but with real relationships between tables (a pantry item row links to the user who owns it, a product row links to a barcode, and so on). On top of the raw database, Supabase bundles:

- **Auth** — handles sign-in (Google OAuth and email/password), issues each session a secure token, and exposes a reliable `auth.uid()` the database can check against.
- **Storage** — holds actual files. This app uses a bucket called `foods` for recipe dish photos and `pantry-images` for user-uploaded item photos.
- **Auto-generated APIs** — the frontend/backend can query tables directly through a typed client, instead of writing a separate API layer for basic reads/writes.

## Row Level Security (RLS)

RLS is a rule written directly onto a database table, enforced by the database itself — not by application code. Even if a request somehow bypassed the backend entirely and hit the database directly, the database would still refuse to return rows that don't belong to that user. Security lives at the data layer, which is a much stronger guarantee than "the app's code remembered to check."

A real example from this project: the `products` table originally had a policy that let *any signed-in user* edit product entries — clearly too permissive, since anyone could tamper with shared product data. The fix was to require a specific role check inside the policy itself:

```
create policy "signed in users can enrich products"
on public.products
for update
using ( has_role(auth.uid(), 'admin') )
with check ( has_role(auth.uid(), 'admin') );
```

`has_role(...)` is a small helper function stored in the database that looks up whether the currently authenticated user (`auth.uid()`) has a given role in a separate `user_roles` table. `USING` controls which existing rows can be touched at all; `WITH CHECK` controls what the row is allowed to look like *after* the change. Together they mean: only an admin can update a product, full stop, enforced by Postgres itself on every single query, from any client.

## Migrations — the database's own version control

Every schema change (a new table, a new column, a new policy) lives as a timestamped SQL file in `supabase/migrations/`. Applied in order, these files fully reconstruct the database structure from nothing — meaning the schema has a complete, reviewable history, the same way git tracks changes to

code.

## Key tables in this project

- `Pantry items` — quantity, purchase date, expiry date, freshness, storage location, all scoped to a `user_id` column checked by RLS on every query.
- `products / pending_products` — the shared, self-learning barcode database, plus a review queue for user-submitted products awaiting admin approval.
- `user_roles` — who has the admin role, separate from the hardcoded owner check.
- `user_settings` — per-user notification toggles (expiry reminders, low stock, etc.), each one actually read by the notification-sending code before anything gets pushed.
- A daily-recipe table with a uniqueness constraint on (user, date) — this is what guarantees exactly one "Tonight's Recommendation" is ever generated per user per day, even under concurrent requests.

## 5. The borrowed-intelligence AI features

---

Photo scanning, receipt OCR, recipe generation, and the assistant chat all work the same underlying way: the backend sends a prompt (plus an image, where relevant) to **Gemini**, through **Lovable's AI Gateway** — a middleman service that forwards the request and attaches the API key server-side, so the key is never exposed to the browser. The model sends back structured JSON that the backend validates and hands to the frontend.

This is *not* a trained model — it's asking an existing, general-purpose AI to interpret something and describe it in a requested format. No weights get updated, nothing is learned; it's borrowed intelligence, used well.

## 6. The self-trained model

---

*This is the part where a model was actually taught something, from data, rather than just asked a question. Everything above this section is about using AI. This section is about training it.*

### The dataset

A labeled Kaggle dataset — *Fruit and Vegetable Disease (Healthy vs Rotten)* — containing photos of 14 different produce types (apple, banana, tomato, potato, and others), each labeled as either healthy or rotten, giving 28 total classes. The images sit in one folder per class, with no pre-made train/test split, so that split was done manually: roughly 80% of the images for training, 20% held out.

### Why transfer learning, not training from zero

Training a vision model completely from scratch requires millions of labeled images and huge amounts of compute — not realistic for a project like this. Instead, this used **transfer learning** with **MobileNetV2**, a model Google already trained on 1.4 million general photos across 1,000 categories (ImageNet). That

pretraining means the model already knows how to see — how to detect edges, textures, colors, and shapes — long before it ever saw a single fruit photo.

*It's the difference between teaching someone to see from birth versus taking someone who already knows how to see, and teaching them the one new specific skill of spotting a bruise on an apple. The second is dramatically faster and needs far less data.*

## Model architecture

MobileNetV2 was loaded with its pretrained weights and its final classification layer removed, then **frozen** — meaning its existing knowledge isn't touched, at first. A new head was added on top, trained from scratch for this specific task:

```
base_model = MobileNetV2(weights='imagenet', include_top=False)
base_model.trainable = False          # freeze the pretrained backbone

x = base_model(input_image)
x = GlobalAveragePooling2D()(x)       # condense feature maps to one vector
x = Dropout(0.3)(x)                  # randomly disable neurons -> less overfitting
x = Dense(128, activation='relu')(x)  # learn task-specific combinations
x = Dropout(0.2)(x)
output = Dense(28, activation='softmax')(x)  # 28 classes, one probability each
```

`GlobalAveragePooling2D` takes the grid of visual features MobileNetV2 extracted and collapses it into a single list of numbers per image. `Dropout` randomly turns off a fraction of neurons during each training step, which forces the network to not over-rely on any single feature — a standard defense against **overfitting** (memorizing the training images instead of learning the general pattern). The final `Dense` layer with **softmax** outputs a probability for each of the 28 classes that all sum to 1 — e.g. 92% confident "banana — rotten".

**Data augmentation** (random flips, small rotations, small zooms) was applied only to training images, artificially creating more visual variety from the same photos so the model doesn't get thrown off by a slightly different angle or crop in real use.

## Training, in two phases

**Phase 1 — train just the new head.** With the MobileNetV2 backbone frozen, only the new layers were trained, using the Adam optimizer at a learning rate of 0.001, for 8 passes over the full training set (each full pass is called an **epoch**). The loss function — `sparse categorical crossentropy` — measures how far off the predicted probability distribution was from the true label; training is the repeated process of nudging the network's internal numbers (**weights**) to gradually reduce that loss.

**Phase 2 — fine-tuning.** The last ~30 layers of MobileNetV2 itself were unfrozen and trained for 5 more epochs, at a much smaller learning rate (0.00001). The smaller rate matters: it lets the pretrained visual features shift slightly to be more specific to produce and rot textures, without a large update wiping out the valuable general knowledge already baked in from ImageNet.

## Evaluation

Accuracy was measured on the held-out test set — images the model never saw during training — since accuracy on training data alone can be misleadingly high. A **confusion matrix** was also generated: a grid where rows are the true class and columns are the predicted class, so the diagonal shows correct predictions and everything off the diagonal shows exactly which classes get mixed up with each other (for example, whether "rotten banana" ever gets confused with "rotten mango").

## Shipping it — TensorFlow.js, running in the browser

The trained Keras model was converted into **TensorFlow.js** format — a `model.json` file plus binary weight-shard files — and placed in the app's public folder. In the browser, the TF.js library loads those files once, and from then on every photo taken in the app's `/scan-my-model` screen is resized and normalized client-side and run straight through the model *on the user's own device*. No network round-trip, no API key, no server cost per scan — genuinely independent inference, not a prompt to someone else's model.

## 7. The full loop, end to end

---

Open the app (**frontend**) → tap scan → the camera captures a photo → the image is sent through one of two paths.

For general product identification, the image goes to the **AI Gateway (backend → Gemini)**. For a freshness or condition read, it can instead be processed directly through the **custom-trained model** running in the browser. These paths are deliberately different: general product identification uses the **backend** because it communicates with an external AI provider, while the custom-trained model has been converted to **TensorFlow.js** and can run directly on the user's device without sending the image to a server.

For the **Gemini path**, the frontend sends the captured image to the appropriate **backend server function**. The backend validates the request, keeps the **AI credentials on the server**, and forwards the image to Gemini through the AI Gateway. Gemini returns structured information such as the **product name, category, estimated shelf life, and freshness-related information**. The backend validates this response before sending it back to the frontend, keeping the AI credentials separated from the browser.

For the **custom-trained model path**, the browser loads the **TensorFlow.js model** and its weight files from the application's public folder. The captured image is **resized and normalized**, then passed through the **MobileNetV2-based network** running on the user's device. The model produces probabilities for its trained classes, allowing the application to determine the predicted freshness or condition **without making a network request or exposing an API key**.

Once the result is produced, the application associates it with the **signed-in user** and stores it in the **Supabase pantry table**. **Row Level Security (RLS)** is applied directly at the database level, so the database verifies which rows the authenticated user can access or modify. This provides an **additional layer of protection** if something goes wrong in the application layer.

After the pantry record is created or updated, **TanStack Query** detects the changed data. The relevant query is **invalidated and refetched**, causing the pantry screen to update without requiring a manual refresh. The complete process therefore connects the **camera, AI processing, custom local inference, backend functions, authentication, database security, and frontend state management** into one continuous workflow.

In one line: the **frontend** captures and displays the user's interaction, the **backend** handles sensitive operations and external AI communication, the **custom model** performs local inference in the browser, **Supabase RLS** protects the database, and **TanStack Query** brings the updated data back into the interface.

## 8. Q&A

---

### Q: What's the difference between your frontend and backend?

A: The frontend is what runs in the user's browser — the UI, built with React and TypeScript. The backend runs on a server and is the only part trusted to hold API keys, talk to the database with elevated permissions, and call external AI services. The frontend calls backend "server functions" almost like normal function calls, but the framework routes them over the network — the user's device never sees a raw database credential or an AI API key.

### Q: Why use transfer learning instead of training a model from scratch?

A: Training from scratch needs millions of labeled images and huge compute to learn basic visual concepts like edges and shapes. MobileNetV2 already learned that from 1.4 million ImageNet photos. Transfer learning reuses that foundation and only trains a small new head — and later fine-tunes a few of the backbone's own layers — specifically for fresh-vs-rotten classification, which needs vastly less data and time while still getting strong accuracy.

### Q: What is Row Level Security and why does it matter?

A: It's a rule enforced by the database itself, not the application code, that restricts which rows a user can see or modify. Even if a request bypassed the app's backend entirely, Postgres would still refuse to return another user's data. It means the security guarantee doesn't depend on every line of app code being written correctly — it's enforced structurally, at the data layer.

### Q: How does your app make sure the AI's daily recipe recommendation doesn't change every time you refresh the page?

A: The daily-recipe table has a database constraint that only allows one row per user per calendar day. The backend function checks the database first: if a recipe already exists for today, it returns that saved one; only if nothing exists yet does it call the AI model and save the new result. That combination of checking-first and a database constraint also prevents duplicate generation if two requests happen to race each other.



**Q: What does 'freezing' a layer mean during training?**

A: It means that layer's internal numbers (weights) are not updated during that phase of training — its gradients are computed but ignored. In this project, MobileNetV2's pretrained layers were frozen at first so only the brand-new classification head learned anything, preserving all the general visual knowledge already baked into the backbone. Later, a small number of its top layers were deliberately unfrozen for fine-tuning, at a very low learning rate, to gently specialize that knowledge toward produce specifically.

**Q: Why does your custom model run in the browser instead of calling an API like the other AI features?**

A: After training, the model was converted to TensorFlow.js format and shipped as static files with the app. The browser loads it once and then runs every prediction locally on the user's device — no network call, no per-request cost, and it keeps working even with a poor connection. It also demonstrates a genuinely independent, self-trained model rather than relying on someone else's hosted AI.

**Q: What would happen if someone tried to call your backend functions without logging in?**

A: Most server functions are wrapped with an authentication middleware that runs before the actual handler. It checks for a valid Supabase session and rejects the request immediately if there isn't one, so the real logic — and any database or AI access — never executes for an unauthenticated caller.

**Q: How do you know your model is actually good, not just memorizing the training images?**

A: By evaluating it on a held-out test set — images it never saw during training at all — rather than only checking accuracy on the training data, which can look artificially perfect if the model has just memorized it. A confusion matrix was also generated on that test set to see specifically which classes get confused with each other, which is more informative than a single accuracy number.